

Generative AI vs Agentic AI

Understanding why agentic AI emerged, by evolving a single HR-hiring chatbot from a plain generative AI assistant into a fully autonomous agent.

WHAT THIS CHAPTER COVERS

- ▶ What generative AI is and how it differs from traditional AI
- ▶ Real-world applications of generative AI
- ▶ The HR-recruiter hiring workflow case study
- ▶ Stage 1: Simple LLM-based chatbot
- ▶ Stage 2: RAG-based chatbot
- ▶ Stage 3: Tool-augmented chatbot
- ▶ Stage 4: Agentic AI chatbot
- ▶ Core differences: generative AI vs agentic AI

Generative AI vs Agentic AI

1. What Is Generative AI?

Definition

Generative AI refers to a class of AI models that can create new content — text, images, audio, code, and video — that resembles human-created data. Rather than analyzing existing data to classify or predict a fixed label, generative models learn to produce entirely new artifacts in a given modality.

Timeline & Key Product Categories

Generative AI has existed as a mainstream, transformative technology for roughly three years. In that time, several product categories have become dominant:

- **LLM-based chatbots** — ChatGPT, Gemini, Claude, Grok — the first true mass-market example of generative AI, capable of human-like text generation.
- **Image generation models** — diffusion-based models such as DALL-E and Midjourney, which turn a text description into an image.
- **Code generation LLMs** — models such as Code Llama, fine-tuned specifically to write software.
- **Text-to-speech (TTS) models** — e.g. ElevenLabs, which convert a text description into natural-sounding human speech.
- **Video generation models** — e.g. Sora, which turn a text description into a short video clip.

REAL WORLD INSIGHT

Generative AI has continued to improve rapidly year over year — early image models produced garbled text inside images, while recent models render legible text reliably. Expect this trend of continuous, fast improvement to continue.

🔄 Quick Revision — What Is Generative AI?

- Generative AI creates new content (text, image, audio, code, video) that resembles human-created data.
- It has existed as a mainstream technology for about three years.
- Key categories: LLM chatbots, image generators, code-generation LLMs, TTS models, video-generation models.
- Its output quality has continuously improved since its introduction.

2. Generative AI vs Traditional AI Systems

How Traditional AI Works

Traditional AI (classical ML and deep learning built before the generative AI era) is trained on labeled data with defined inputs and outputs. Its job is to find the mathematical relationship between input and output so that, given a new input, it can predict an output.

- **Classification example:** given an incoming email, predict whether it is spam or not spam. Given a chest X-ray, predict whether the patient has cancer.
- **Regression example:** given past data, predict a continuous value such as tomorrow's temperature or a company's stock price.

How Generative AI Is Different

Generative AI does not try to learn an input-output mapping. Instead, it learns the **distribution** of the training data — essentially, the underlying "nature" of the data. For example, if a generative model is trained on thousands of cat images, it learns what a cat looks like in the real world. Once it understands this distribution, it can sample a brand-new image from within that distribution — producing a new, realistic image of a cat that never existed before.

IMPORTANT

Traditional AI learns a mapping from input to output for prediction. Generative AI learns the distribution of the data itself so it can generate new samples from that distribution.

Aspect	Traditional AI	Generative AI
Goal	Learn input → output relationship	Learn the distribution of the data
Typical tasks	Classification, regression	Content generation (text, image, audio, video, code)
Output	A label or a numeric prediction	A new, original artifact resembling human-created data

🔄 Quick Revision — Generative AI vs Traditional AI

- Traditional AI learns input-output relationships to classify or predict values.
- Generative AI learns the underlying data distribution to generate new samples.
- Classification and regression are classic traditional-AI problem types.
- Content generation across modalities is the hallmark of generative AI.

3. Applications of Generative AI

Generative AI's defining strength is that it can mimic human creativity convincingly enough that its output feels human-made. This has driven rapid adoption across industries.

1. CREATIVE & BUSINESS WRITING

Drafting blog posts from an outline, polishing business emails for tone and grammar, and summarizing or replying to emails (e.g. Gmail's built-in generative features).

2. SOFTWARE DEVELOPMENT

Auto-completion tools that predict and generate entire blocks of code, and chat-based debugging where an error message is pasted into an LLM to diagnose the cause.

3. CUSTOMER SUPPORT

Large companies (Ola, Uber, Zomato, Swiggy) deploy generative-AI chatbots to resolve common customer queries automatically, escalating to a human executive only when the bot cannot resolve the issue.

4. EDUCATION

Clarifying doubts while watching a lecture, building a personalized learning curriculum, and simplifying or summarizing a difficult topic on demand.

5. DESIGN

Generating thumbnails, infographics, and short ad clips (using tools like Sora and Runway) from a text description instead of creating them manually.

BEST PRACTICE

When evaluating a new generative AI product idea, identify which specific modality (text, image, audio, video, code) and which specific repetitive creative task it will replace or accelerate — broad claims of "AI-powered" rarely map to a concrete use case.

Quick Revision — Applications of Generative AI

- Major application areas: creative/business writing, software development, customer support, education, design.
- Adoption is driven by generative AI's ability to mimic human creativity convincingly.
- Customer support chatbots typically escalate unresolved queries to a human executive.
- Design use cases include thumbnails, infographics, and short video ad clips.

4. Case Study Setup: The HR Recruiter Problem

To build intuition for why agentic AI is needed, the lecture walks through one continuous case study: an HR recruiter tasked with hiring a backend engineer. The end-to-end hiring task breaks down into seven steps.

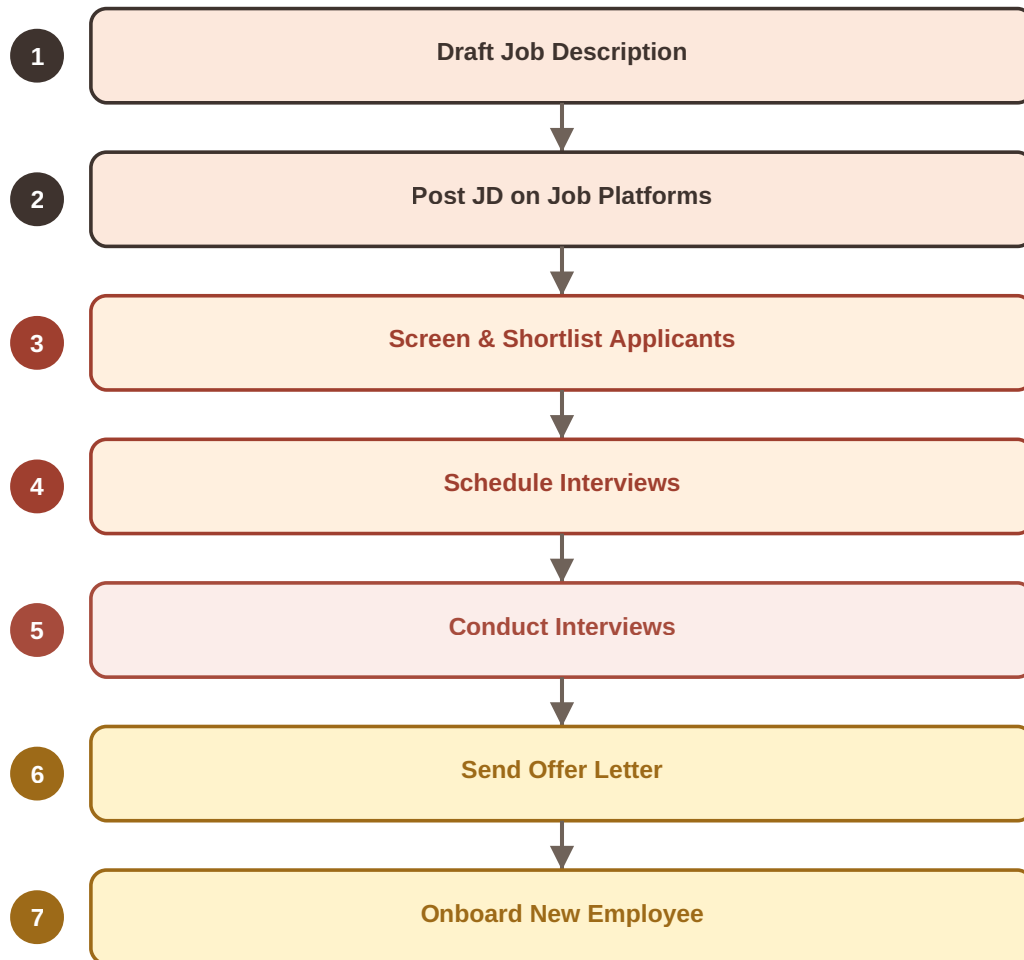


Figure 1.1 — The seven-step HR hiring workflow used throughout this chapter's case study.

This same seven-step workflow is used across all four evolution stages in this chapter so the improvements at each stage can be compared directly. Each stage below re-runs this workflow using a progressively more capable AI system.

Quick Revision — Case Study Setup

- The case study follows an HR recruiter hiring a backend engineer.
- The workflow has seven steps: draft JD, post JD, shortlist, schedule, interview, offer, onboard.
- The same workflow is reused across all four chatbot evolution stages for direct comparison.

5. Stage 1: Simple LLM-Based Chatbot

How It Works

In this first stage, the recruiter uses a plain LLM-based chatbot — no company data, no tools, just a general-purpose conversational model. For every step of the hiring workflow, the recruiter manually prompts the chatbot, reads its reply, and manually executes the resulting action themselves (e.g. posting the JD on LinkedIn, or emailing a candidate).

Example

The recruiter asks for a JD for a backend engineer with 2–4 years of experience. The chatbot drafts one instantly. Later, the recruiter asks where to post it; the chatbot suggests LinkedIn and Naukri based on its general training knowledge. For shortlisting, the chatbot offers generic advice (e.g. "look for Python and cloud experience") rather than company-specific guidance.

COMMON MISTAKE

Assuming a plain LLM chatbot understands your company's context. It only has general training knowledge — its JD, shortlisting advice, and interview questions are generic and not tailored to any specific organization.

🔄 Quick Revision — Simple LLM Chatbot

- The chatbot has no company-specific knowledge and no tool access.
- The human manually executes every action (posting, emailing, scheduling) after the chatbot drafts content.
- Advice given (shortlisting criteria, interview questions) is generic, not company-specific.

PROBLEMS WITH THE SIMPLE LLM CHATBOT

1. **Reactive, not proactive** — the human drives every step; the chatbot only reacts to prompts and never initiates the next action on its own.
2. **No memory / context awareness** — the chatbot forgets prior conversations; the recruiter must re-explain context (e.g. re-paste the JD) each time.
3. **Generic advice** — responses are not tailored to the specific company's hiring practices, salary bands, or past templates.
4. **No ability to take action** — the chatbot can draft a JD or an email but cannot post it, send it, or otherwise act in the world; the human must execute everything manually.

6. Stage 2: RAG-Based Chatbot

Problem It Solves

To fix the "generic advice" problem, the chatbot is connected to the company's own knowledge base using *Retrieval-Augmented Generation (RAG)*. The recruiter feeds the chatbot past JD templates, the company's hiring playbook, internal salary bands, shortlisting criteria, past interview question banks, and onboarding document templates (offer letters, welcome emails, employee policies).

How It Works

Company documents are ingested once and stored as embeddings in a vector store. At query time, the chatbot retrieves the most relevant chunks of company knowledge for the current question and uses them, alongside the user's query, to generate a company-specific (rather than generic) response.

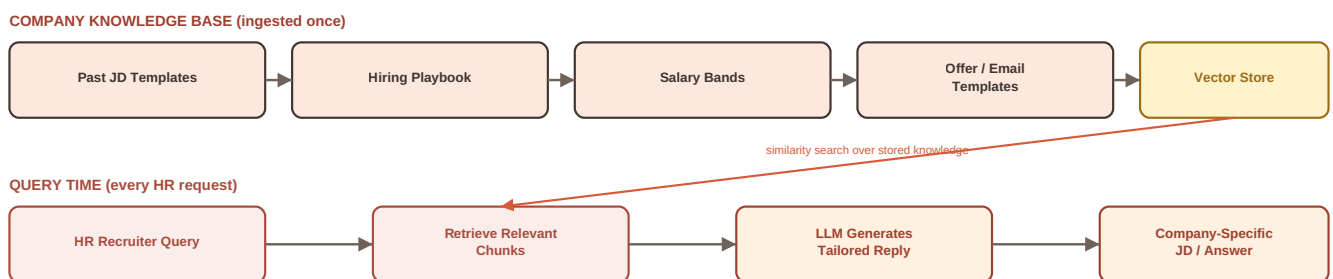


Figure 1.2 — RAG chatbot architecture: company knowledge is ingested once and retrieved at every query to tailor responses.

Example

When asked for a JD, the RAG chatbot now automatically infers the tech stack (Python, Django) and the correct salary band for 2–4 years of experience from the company's own historical data — without the recruiter having to specify these details explicitly. Shortlisting advice, interview questions, and offer letters are all now drawn from the company's own past hiring history and templates.

INTERVIEW TIP

If asked to explain RAG in an interview, emphasize the two-phase structure: an offline ingestion phase (documents → chunks → embeddings → vector store) and an online query phase (query → retrieve relevant chunks → augment the prompt → generate a grounded answer).

Quick Revision — RAG-Based Chatbot

- RAG = Retrieval-Augmented Generation: retrieving relevant company documents to ground LLM responses.
- Ingested knowledge: JD templates, hiring playbook, salary bands, shortlisting criteria, past interview questions, offer/onboarding templates.
- Responses become company-specific instead of generic.
- Ingestion happens once; retrieval happens on every query.

PROBLEMS STILL REMAINING AFTER RAG

Solved: generic advice → now the chatbot gives company-specific, tailored responses.

Still unsolved:

1. **Reactive, not proactive** — the human still drives every step.
2. **No memory / context awareness** — the chatbot still forgets earlier conversations.
3. **No ability to take action** — it can draft a company-specific JD or email, but still cannot post it or send it itself.

7. Stage 3: Tool-Augmented Chatbot

Problem It Solves

To fix the "cannot take action" problem, the RAG chatbot is integrated with external tools/APIs, so it can act on the world rather than only reply with text.

Tools Integrated

- **LinkedIn / Naukri API** — post job openings and track applications directly.
- **Resume parser tool** — ingest and understand PDF resumes.
- **Calendar API** — check availability and schedule interviews.
- **Email API** — send and receive emails.
- **HR Management Software** — trigger the full onboarding workflow.

Example

The chatbot can now post the JD directly to LinkedIn and Naukri via API, check application counts on its own when asked, download and screen resumes using the resume parser, check the recruiter's calendar and send interview invites, and send the offer letter and trigger onboarding directly — instead of merely drafting content for the human to send.

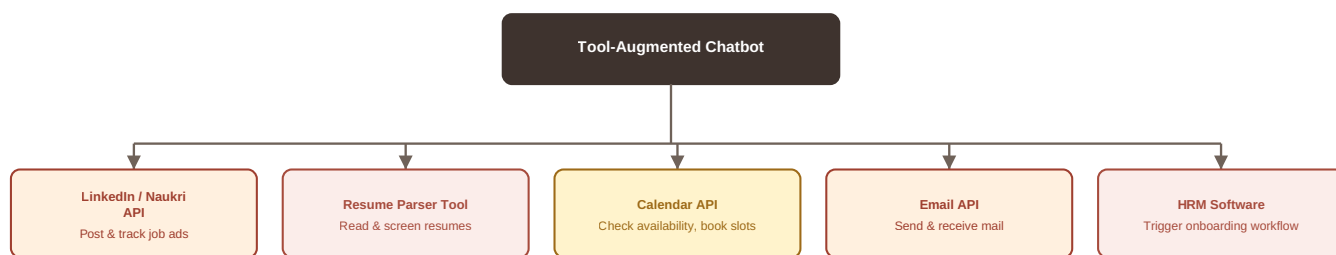


Figure 1.3 — Tool-augmented chatbot: the same RAG chatbot gains the ability to act via external tool integrations.

COMMON MISTAKE

Confusing a tool-augmented chatbot with a fully agentic system. Tool access alone does not make a system proactive — the human still has to instruct it to use each tool at each step.

🔄 Quick Revision — Tool-Augmented Chatbot

- Tool augmentation connects the chatbot to external APIs: job portals, resume parser, calendar, email, HRM software.
- This solves the 'cannot take action' problem — the chatbot can now act, not just reply.
- The human must still explicitly instruct the chatbot to use each tool at each step.

PROBLEMS STILL REMAINING AFTER TOOL AUGMENTATION

Solved: the chatbot can now take real actions via its integrated tools.

Still unsolved:

1. **Reactive, not proactive** — the human still initiates and drives each step.
2. **No context awareness / memory** — still no persistent memory across steps.
3. **Cannot adapt on its own** — even when the recruiter points out a problem (e.g. too few applications), the chatbot only reacts to being told; it never identifies the problem or proposes a strategy on its own initiative.

8. Stage 4: Agentic AI Chatbot

Problem It Solves

The final evolution makes the chatbot *proactive*, *context-aware*, and *adaptable* — the three defining traits of an AI agent. Given only a high-level goal, it plans the full workflow itself, executes each step using its tools, remembers everything it has already done, and adapts its strategy when something goes wrong — asking the human only for approval at key checkpoints.

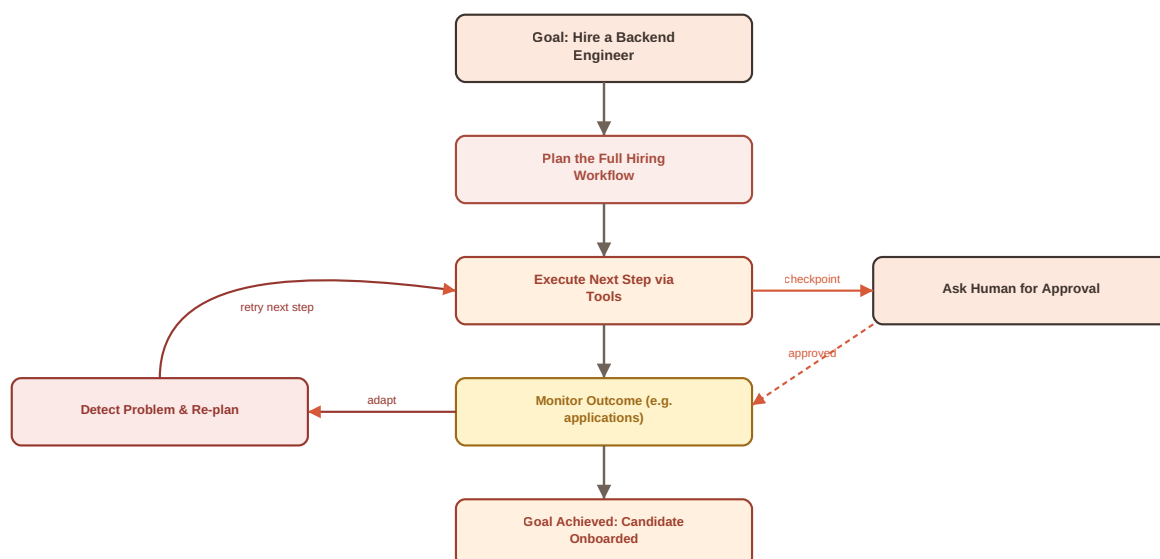


Figure 1.4 — The agentic loop: a single goal triggers autonomous planning, execution, monitoring, and adaptation, with the human only approving checkpoints.

How It Works

1. The recruiter states only the end goal: "I want to hire a backend engineer."
2. The agent independently plans the entire workflow (draft JD → post → monitor → screen → interview → offer → onboard).
3. It executes the plan step by step, using its tools, and checks in with the human only for approvals (e.g. reviewing the JD, confirming an interview slot).
4. It continuously monitors outcomes — for example, noticing that only two applications came in against expectations.
5. It adapts on its own: identifying the low-application problem, proposing a fix (broaden the JD to "full-stack engineer," boost the LinkedIn post), and executing the fix once approved.

BEST PRACTICE

Design agentic systems with explicit human-in-the-loop checkpoints (e.g. approving a JD or an offer letter) rather than full autonomy everywhere — this preserves oversight while still removing the bulk of manual execution work.

Quick Revision — Agentic AI Chatbot

- An AI agent is proactive, context-aware (has memory), and adaptable.
- Given only a goal, the agent plans and executes the full multi-step workflow itself.
- The human's role shrinks mostly to approving key checkpoints.
- The agent can detect problems on its own (e.g. low application count) and propose/execute a fix.

ALL FOUR PROBLEMS NOW SOLVED

Proactive: the agent identifies goals and next steps on its own instead of waiting to be told.

Context-aware: memory lets it track what it has already done and what remains.

Specific advice: retained from the RAG stage — still grounded in company data.

Can take action: retained from the tool-augmented stage — still executes via integrated tools.

Adaptable: new in this stage — the agent detects problems and re-strategizes without being told to.

9. Generative AI vs Agentic AI — Core Differences

Aspect	Generative AI	Agentic AI
End goal	Produce a piece of content (text, image, video, code)	Achieve a stated goal, whatever it takes
Interaction style	Reactive — responds only when prompted at each step	Proactive — plans and executes autonomously
Human's role	Guides every single step	Mostly approves checkpoints; agent does the rest
Relationship	A capability / building block	A behavior built on top of generative AI (planning, tools, memory)

IMPORTANT

Agentic AI is not a replacement for generative AI — it is a superset that uses generative AI (LLMs) as its reasoning and planning engine, combined with tools and memory. As commonly phrased: "**Generative AI is a capability, whereas agentic AI is a behavior.**"

🔄 Quick Revision — Generative AI vs Agentic AI

- Generative AI's end goal is content creation; agentic AI's end goal is achieving a stated goal end-to-end.
- Generative AI is reactive; agentic AI is proactive and autonomous.
- In agentic AI, the human mainly provides approvals rather than driving every step.
- Generative AI is a building block (capability) that agentic AI is built on top of (behavior).

K. Key Takeaways

- **Generative AI** creates new content by learning the distribution of training data, unlike traditional AI which learns input-output mappings for prediction.
- **Generative AI applications** span writing, software development, customer support, education, and design.
- **The HR hiring case study** shows the same seven-step workflow improving across four AI system stages.
- **A simple LLM chatbot** is reactive, has no memory, gives generic advice, and cannot take action.
- **A RAG-based chatbot** solves the generic-advice problem by grounding responses in company knowledge.
- **A tool-augmented chatbot** solves the no-action problem by connecting to external APIs (LinkedIn, calendar, email, HRM software).
- **An agentic AI chatbot** solves the remaining problems by being proactive, context-aware, and adaptable — planning and executing an entire goal with only human checkpoints.
- **Generative AI is a capability; agentic AI is a behavior** built on top of generative AI using planning, memory, and tools.

R. Revision Sheet

Generative AI

A class of AI models that create new content by learning the distribution of training data, across text, image, audio, code, and video.

Traditional AI

Classical ML/deep learning that learns an input-output mapping for classification or regression tasks.

RAG (Retrieval-Augmented Generation)

Grounding an LLM's responses in retrieved chunks of an organization's own documents, via an ingestion phase and a query-time retrieval phase.

Tool-Augmented Chatbot

An LLM/RAG chatbot connected to external APIs (job portals, calendar, email, HRM software) so it can take real actions, not just generate text.

AI Agent

A system that is proactive, context-aware (has memory), and adaptable — it plans and executes toward a goal with minimal human guidance.

Agentic AI

A behavior built on generative AI, combining LLM reasoning, planning, memory, and tool use to autonomously achieve a stated goal.

I. Interview Questions

- Q1.** How does generative AI fundamentally differ from traditional AI in terms of what it learns from data?
- Q2.** What is the difference between a classification problem and a regression problem in traditional AI, with examples?
- Q3.** What is Retrieval-Augmented Generation (RAG), and what two phases does a RAG pipeline consist of?
- Q4.** Why does a plain LLM-based chatbot give "generic" advice, and how does RAG fix this?
- Q5.** What does it mean for a chatbot to be "tool-augmented," and what problem does this solve that RAG alone does not?
- Q6.** Define the three defining traits of an AI agent: proactive, context-aware, and adaptable.
- Q7.** Explain, with the HR hiring example, how an agentic AI system would handle a sudden drop in job applications differently from a tool-augmented chatbot.
- Q8.** What is the relationship between generative AI and agentic AI? Is one a subset of the other?
- Q9.** What role does the human play in an agentic AI workflow, compared to a simple generative AI chatbot workflow?
- Q10.** Why is memory/context awareness necessary for a system to be considered agentic?

F. Further Reading

- OpenAI — documentation on retrieval-augmented generation and function/tool calling.
- LangChain documentation — RAG pipelines, retrievers, and vector stores.
- LangGraph documentation — building stateful, agentic workflows (covered in upcoming chapters of this series).
- Anthropic — "Building Effective Agents," a practical overview of agent design patterns.